

Big Data and Large-Scale Computing

CSE412

Shajulin Benedict
shajulin@iiitkottayam.ac.in

Indian Institute of Information Technology Kottayam
www.sbenedictglobal.com

Exam Pattern

- Theory : 100 (Scaled down to 30)
 - Mid Exam : 25
 - One Quiz : 10
 - Two assignments : 20
 - End Exam : 45
- Lab : 30
 - Continuous Evaluation : 15
 - Final Exam : 15

Objectives

To know the fundamental concepts of big data and analytics.

To provides an in-depth understanding of terminologies and the core concepts behind big data problems.

To introduce some of the most common frameworks such as Apache Spark, Hadoop, MapReduce.

To understand large scale data storage technologies such as in-memory key/value storage systems, NoSQL distributed databases, Apache Cassandra, HBase.

To introduce various Big Data Streaming platforms such as Apache Spark Streaming, Apache Kafka Streams that has made big data analysis easier and more accessible.

Syllabus

- Evolution of Big data - Best Practices for Big data Analytics - Big data characteristics - Characteristics of Big Data Applications - Big Data Modelling - Understanding Big Data Storage - A General Overview of High Performance Architecture - HDFS – MapReduce and YARN - Map Reduce Programming Model.
- PIG - PIG Latin - Input, Output- Relational Operators - User Defined Functions – Join Implementations - Integrating Pig with Legacy Code and Map Reduce - Developing and Testing Pig Latin Scripts - Embedding Pig Latin in Python.

Syllabus

- Introduction to HIVE - Data Types and File Formats - Databases in Hive - HiveQL: Data Definition - Data Manipulation - Queries - Views - Indexes - Schema Design
- Introduction to Streams Concepts - Stream Data Model and Architecture -Introduction to Big Data Streaming Platforms for Fast Data - Introduction to Big Data Streaming Systems - Big Data Pipelines for Real-Time computing - Introduction to Spark Streaming – Kafka - Streaming Ecosystem.

Books

- Tom White , Hadoop: The Definitive Guide, 4th edition
O'Reily Publications, 2015.

Big Data and Large Scale Computing



Big Data and Large Scale Computing

- Big data is defined as *a large amount of data that can't be stored or processed by conventional data storage or processing equipment.*
- The data are *so complex and expansive* that they cannot be interpreted by humans nor fit into a relational database for analysis.

Big Data and Large Scale Computing - Overview

- Big Data

- Human-generated (e.g., emails, tweets)
- Machine-generated (e.g., IoT)
- Business-data (e.g., transactions)

- Large scale computing

- Typically operated over thousands of servers.



Big Data and Large Scale Computing - Overview

- How big is big data?

Category	Data Size Estimate	Example Use Case
Gigabytes (GB)	10^9 bytes (~1 billion)	Small business CRM or e-commerce logs
Terabytes (TB)	10^{12} bytes (~1 trillion)	Retail chain's daily transaction logs
Petabytes (PB)	10^{15} bytes (~1 quadrillion)	Netflix storing thousands of HD/4K movies Facebook processes 4 petabytes per day
Exabytes (EB)	10^{18} bytes	Global data created daily (~400 EB/day, 2025 est.) Youtube uploads over 1EB data daily
Zettabytes (ZB)	10^{21} bytes	Global cloud data storage by 2025: ~200 ZB+
Yottabytes (YB)	10^{24} bytes	Hypothetical — beyond current infrastructure

Evolution of Big Data

- Pre-BigData period (Before 2000s)
 - Structured data, RDBMS, SQL, (for small retail shops)
- Web/Social Media Impacted era (2000-2010)
 - Hadoop, NoSQL or semi-structured,
- Big Data 2.0 (2010 – 2020)
 - Real-time analytics, ML-based
 - Spark, Kafka, Hive, Flink, Storm
- Intelligent Big Data (2020 – 2025)
 - Edge+AI, Federated with privacy
- 2025+ period
 - LLMs (for personalization)
 - Quantum supported options
 - Digital Twin supported options



Characteristics of Big Data

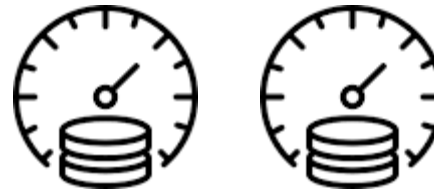
Volume

Size of data



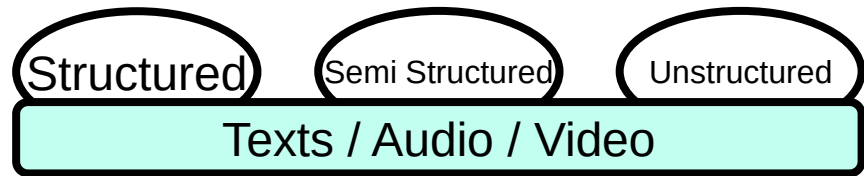
Velocity

Rate of data accumulation



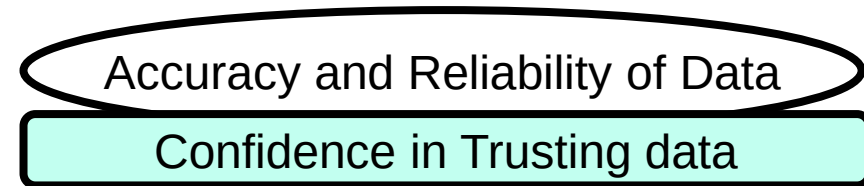
Variety

Diff. sources of data



Veracity

Quality of data



Value

Operational value - insights

Characteristics of Big Data

Variability

Measure of inconsistencies

Parotta tastes differently every day

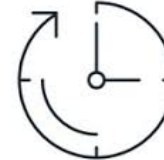
(Valid)-ity

The degree to which data is accurate and correct. (Trustworthiness)

Similar to veracity.

Volatility

How long the data becomes relevant or usable? (Lifespan)



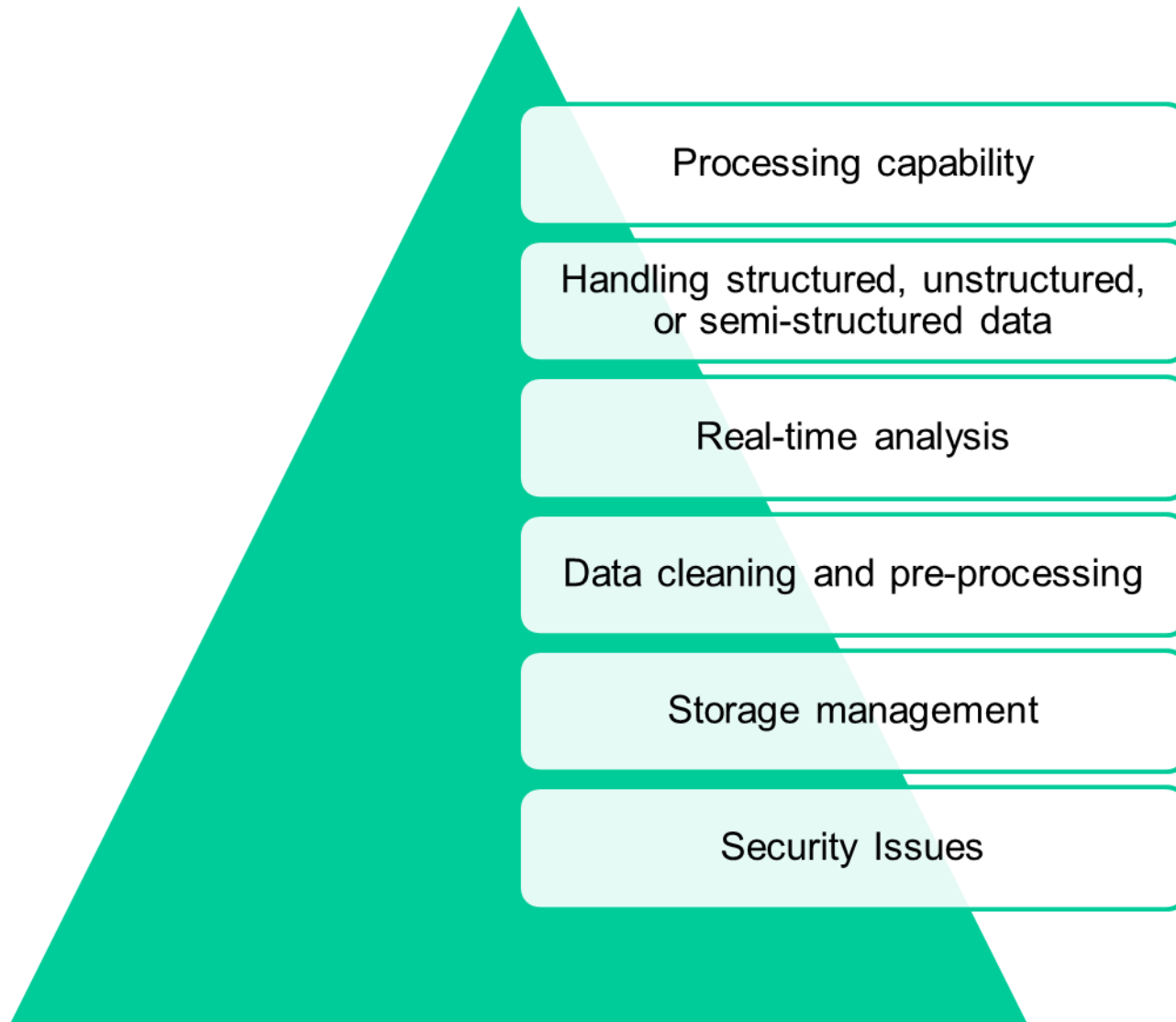
Vulnerability

Harm in sharing data

Visualization

Providing patterns

Challenges of Big Data



- Data modeling is the structure of databases. Also, it defines the relations and storage methods.
- It is important to query peta/zeta bytes of data. (Otherwise, it is slower).
- Traditional data modeling approaches
 - Relational model
 - Collection of tables (record-based approach)
 - Entity-Relationship model
 - Collection of entities and relationships (to design databases)
 - Object-based data model
 - Representation of E-R model using OOPs concepts
 - Semi-structured data model
 - Partially structured data (used while transferring data between devices – e.g., XML format or JSON format data)

Other Aspects – for Data Modeling



No-SQL and Schema-less



Distributed and Scalable features



Time series and Streaming data



Data Lakes and Lakehouse



ML-based Data Modeling



Data Governance and Metadata Modeling

No-SQL and Schema-less Modeling

- No-SQL

- Unstructured or semi-structured data
- It has a horizontal scaling feature.
- Flexible schema designs are possible.
- Types of No-SQL
 - Key-value – e.g., DynamoDB
 - Document approach – e.g., mongoDB
 - Column-Family – e.g., Cassandra, Hbase
 - Graph – e.g., Neo4j

- Schema-less

- i.e., no fixed table schema for modeling data.
- i.e., each record can have different fields or structure.

- Adv.

- Agility, scalability, and simplicity

Distributed and Scalable Features

- Here, data modeling is focused on partitioning across multiple servers.
- It targets parallelism, fault tolerance, and data availability.
- Key Concepts include:
 - **Sharding**: Dividing data into parts (shards) and storing each on a different node.
 - **Replication**: Duplicating data across nodes for high availability.
 - **Data locality**: Placing computation near where data resides (e.g., Hadoop, Spark).
- Key examples
 - MongoDB, Hive, or Cassandra

Time series and Streaming Data

- Time series data is a sequence of data points indexed (or timestamped) in a particular order.
- Applications:
 - IoT sensor data
 - Health check data
- Tools or databases based on time series
 - E.g., InfluxDB or TimeScaleDB

Data Lakes and Lakehouse

- A Data Lake is a centralized repository that stores raw, unstructured, semi-structured, and structured data at any scale.
 - Characteristics/Features:
 - Handles flexible formats
 - Highly scalable
 - Uses Cloud-assisted technologies (such as S3 buckets)
- A Lakehouse is a combination of datalakes with the management efforts of data warehouses.
 - E.g, Snowflake, Google BigLake

ML-based Data Modeling

- The data modeling is supported using ML techniques.
- This enables
 - data-driven learning of structure, patterns, trends, and relationships
- Adv:
 - Automated pattern discovery
 - i.e., Learns trends, clusters, or correlations without prior rules
 - Adaptive
 - i.e, Updates models as new data arrives (e.g., drift-aware)
 - High-dimensional modeling
 - i.e, Handles complex, non-linear relationships across many variables
 - Supports unstructured data
 - i.e, Works with text, images, logs, sensor data, etc.

Data Governance and Metadata Modeling

- Data governance

- A framework of policies, processes, roles, and tools to ensure high data quality, privacy, security, and compliance throughout the data lifecycle.

- Metadata modeling

- The practice of designing and managing *data about data* – describing its structure, source, format, meaning, and relationships.

Big Data Life Cycle

- Data Generation – Acquisition

- Data production via. sensors or others

- Data Ingestion

- Raw data is submitted to a system via. batch or real-time pipelines. (e.g., AWS Kinesis)

- Data Storage

- Store them using different architectures (e.g., NoSQL, HDFS, MongoDB, S3)

- Data processing

- Cleaning, Transforming, Removing duplicates, or so forth.

Big Data Life Cycle

- Data Modeling and Analysis

- Different data modeling approaches are applied via. Hive or ML frameworks

- Data Visualization

- Present the insights using dashboards or reports. (E.g., Tableau, Grafana)

- Data Archival & Retention

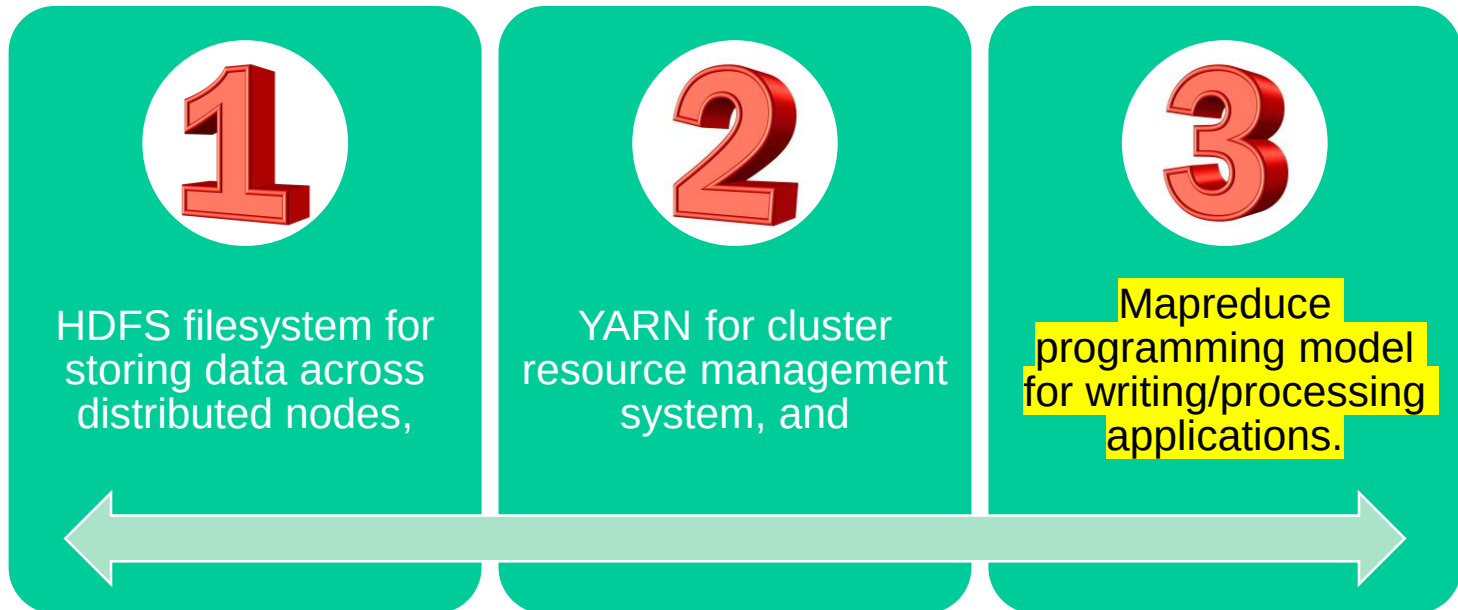
- Crucial data are stored for enabling historical trends. (E.g., Glaciers and backups)

- Data Governance and Security

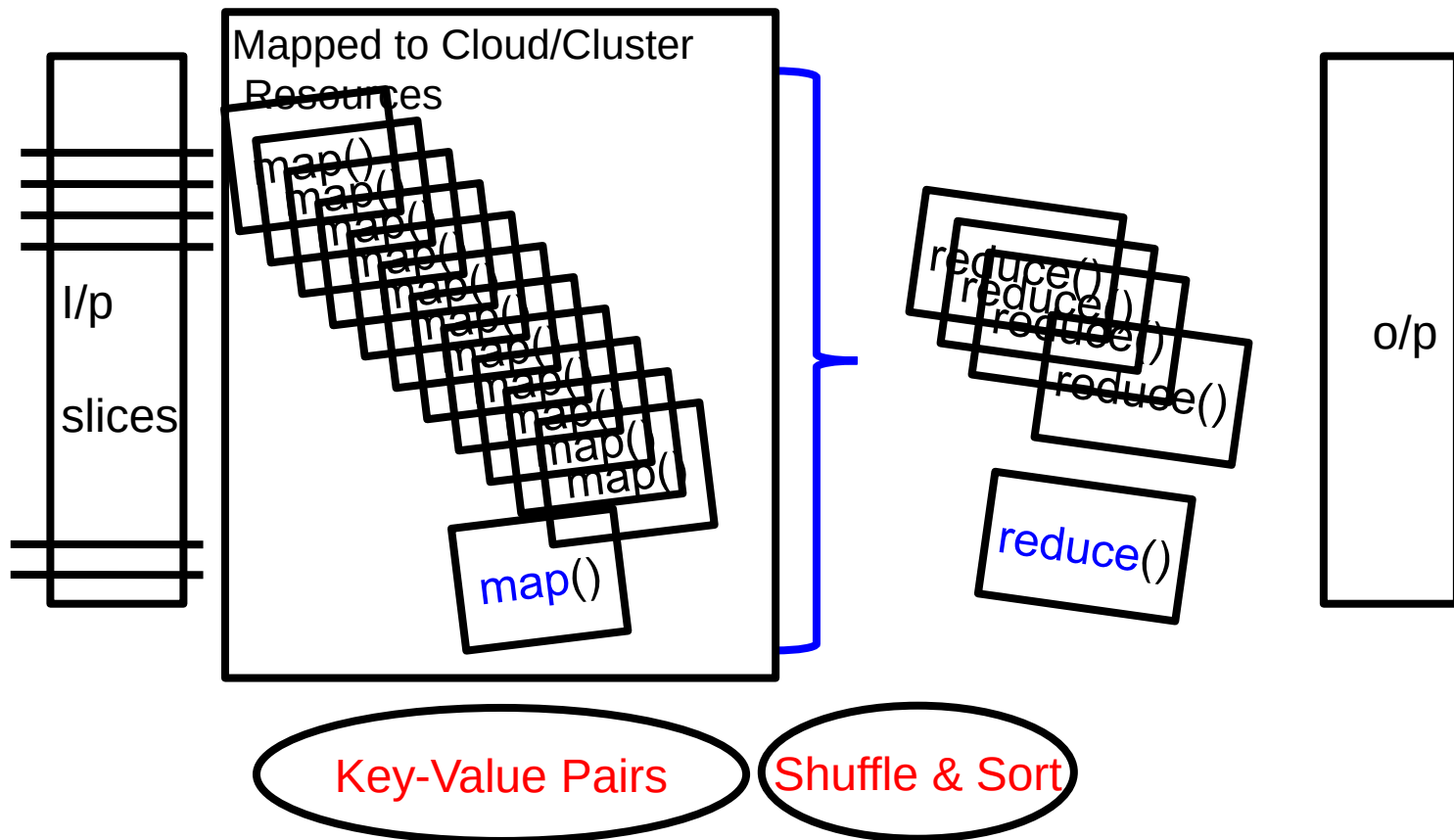
- Applying policies and access control mechanisms (e.g., masking personal identifier details).

Hadoop Fundamentals

- Hadoop is a software library-based framework that allows distributed processing of large datasets across a large computing cluster.
- It utilizes 3 components...



MapReduce Programming Model



MapReduce Programming Model

Phase 1

Splitting data

Phase 2

Apply map() to each chunk of data

Phase 3

Sort and Shuffle (to distribute them to reducers)

Phase 4

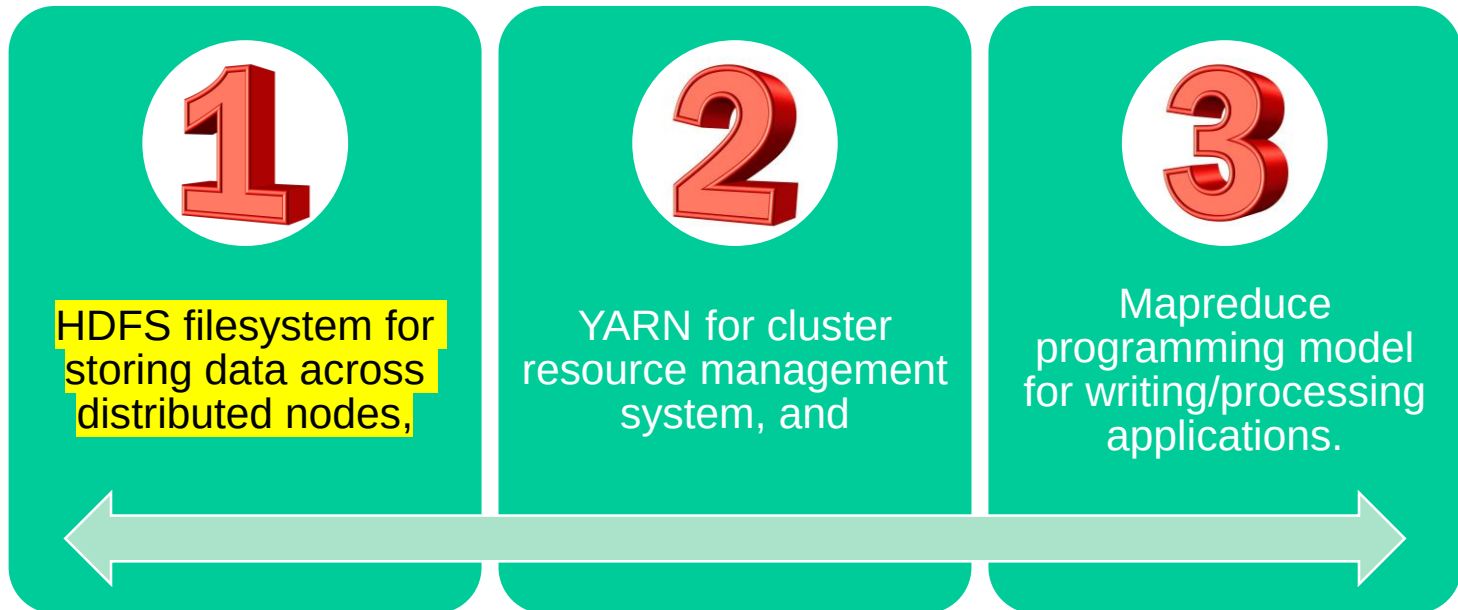
Apply reduce() to produce output.

Important points

- The same map (and reduce) function is applied to all the chunks in the data.
- The map and reduce computations can be carried out in parallel because they're **completely independent** from one another.
- The split is not the same as the internal partitioning into blocks.
- The shuffling and sorting phase is often the most costly component in a MapReduce job.

Hadoop Fundamentals

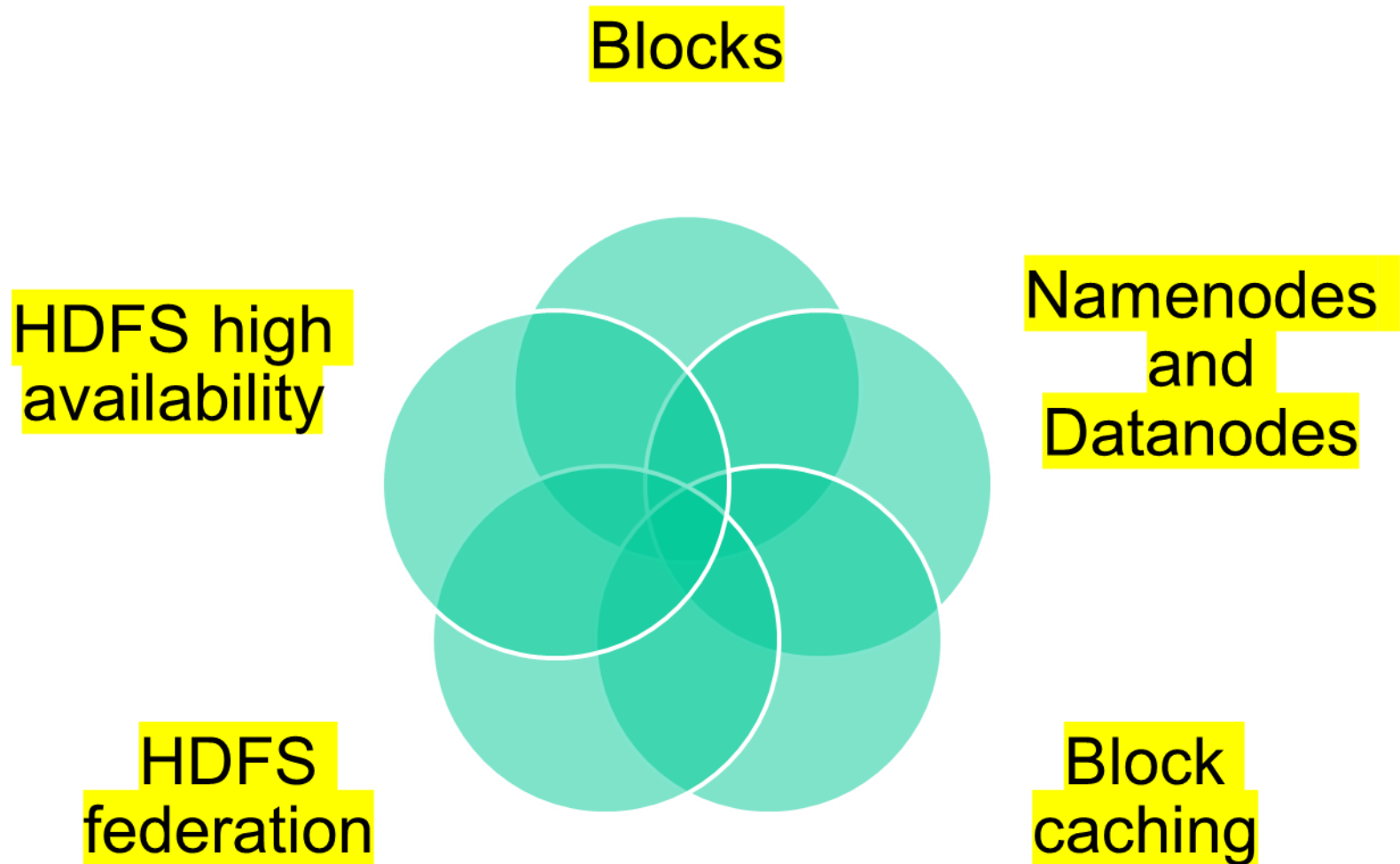
- Hadoop is a software library-based framework that allows distributed processing of large datasets across a large computing cluster.
- It utilizes 3 components...



Distributed File System

- File system -- A file system is a method an operating system uses to store, organize, and manage files and directories on a storage device.
- **Distributed File System** – A file system that manage the storage across a network of machines is called distributed file system.
- HDFS – HDFS is a filesystem designed for ...(3 major points)
 - storing very large files
 - with streaming data access patterns (i.e., write once and read many times patterns)
 - running on clusters of commodity hardware (i.e., doesn't require expensive hardware with an understanding that the hardware failures are possible).

Hadoop's HDFS Concepts



Blocks – In Normal Linux-based System

- In Linux, block devices are handled by a dedicated layer **within the kernel**. This kernel layer
 - manages data transfer in blocks,
 - optimizes storage access and
 - provides an abstraction between the operating system and storage hardware.
- When an application needs to read or write data, it interacts with the file system via. the kernel layer to access blocks (typically 512 bytes).
- These requests are encapsulated into *struct bio structures*, which contain information about
 - the data to be transferred,
 - the target device (SSD or harddrive), and
 - the operation (read or write).

Blocks – In HDFS system

- HDFS follows block-based data accessing approach (i.e., the files are stored as HDFS blocks/chunks).
- The size of a HDFS block is 128 MB (by default).
- These chunks are stored on multiple cluster nodes (not on a single device).
- Why the block size is larger in HDFS?
 - Mainly to optimize the performance for large-scale data processing.
 - In general, the seek time is slower than the network transfer time.
 - Also, the number of map and reduce tasks can be minimized.

Block Abstraction in HDFS

Benefits

- Easy to figure out the number of blocks which can be stored on each disk.
- A file can be larger than any single disk in the network. Hence, block abstraction can figure it out.
-
- Replicas are manageable if abstractions are enabled in HDFS.

Namenodes and Datanodes

- HDFS cluster has two types of nodes
 - Namenodes (master node)
 - Datanodes (worker node)
 - These nodes function by understanding heartbeats between them.
- Namenode
 - It maintains the filesystem tree.
 - It holds the metadata for all the files and directories in the tree.
 - The information is stored persistently on the local disk using two files: a) the namespace image and b) edit log
 - It knows the datanodes on which all the blocks are located.
 - But, it doesn't store the information about the block locations persistently.

Namenodes and Datanodes

- Namenodes (contd...)

- The block locations are reconstructed from datanodes when the system starts.
- Without namenodes, the filesystem cannot be used.
- Hence, namenodes have to be resilient to failures.
 - Either via. periodic backups or providing secondary namenode

- Datanodes

- They are the workhorses (workers) of the filesystem.
- These nodes read blocks from disks.

Block Caching

- Block caching

- To frequently access files, the blocks are explicitly cached in datanodes.
- Caching is necessary to improve the performance of HDFS-based applications.

HDFS Federation

- We know that namenode keeps a reference to every files and blocks in the filesystem.
- In some cases, it is better to have multiple namenodes (a federation approach).
- In doing so, each namenode can represent a file root tree...
 - E.g., /user → corresponds to one namenode
 - /share -> corresponds to second namenode
 - /etc → corresponds to another namenode, and so forth.

HDFS High Availability

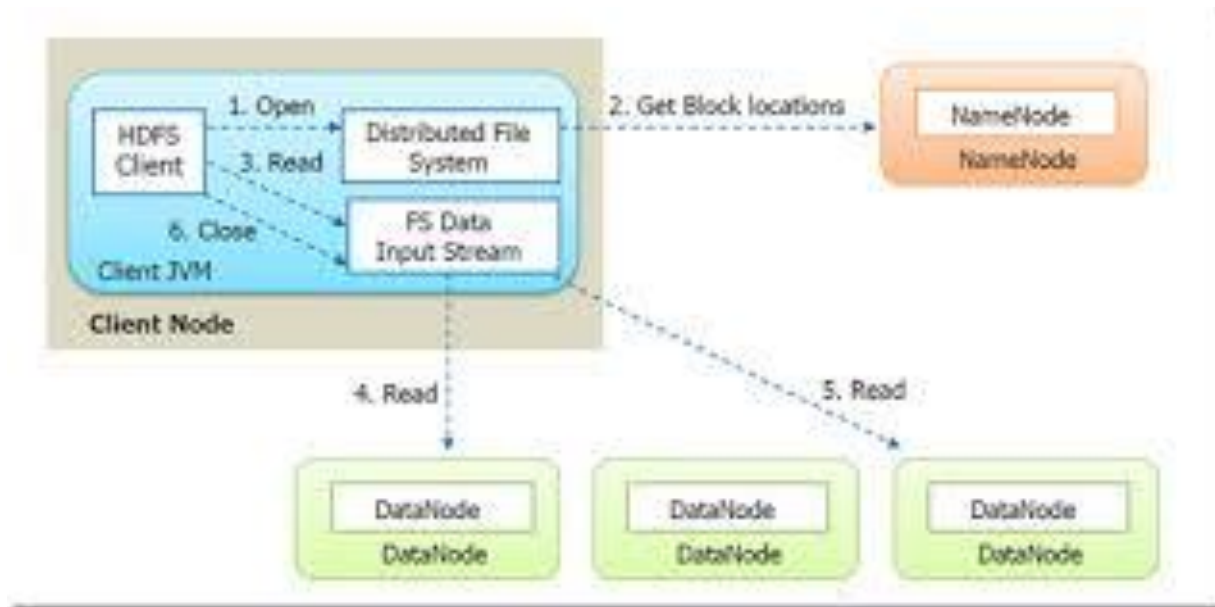
- It is better to have a pair of namenodes for high availability (i.e., creating replicas).
- The replica of the namenode in HDFS filesystem is named as secondary namenode.
- The secondary namenode acts in standby mode (which functions when failure occurs in the active node).
- The transition from the active namenode to the secondary namenode is managed by an entity named as *failover controller*.
- The failover controller can be initiated either using i) Zookeeper or ii) Manually (also, named as graceful failover)

HDFS High Availability

- Choice 1 – Using an NFS filer (via. NFS).
- Choice 2 – Using a Quorum Journal Manager. This provides a highly available edit log for all machines.
 - e.g., Zookeeper follows this approach for high availability.

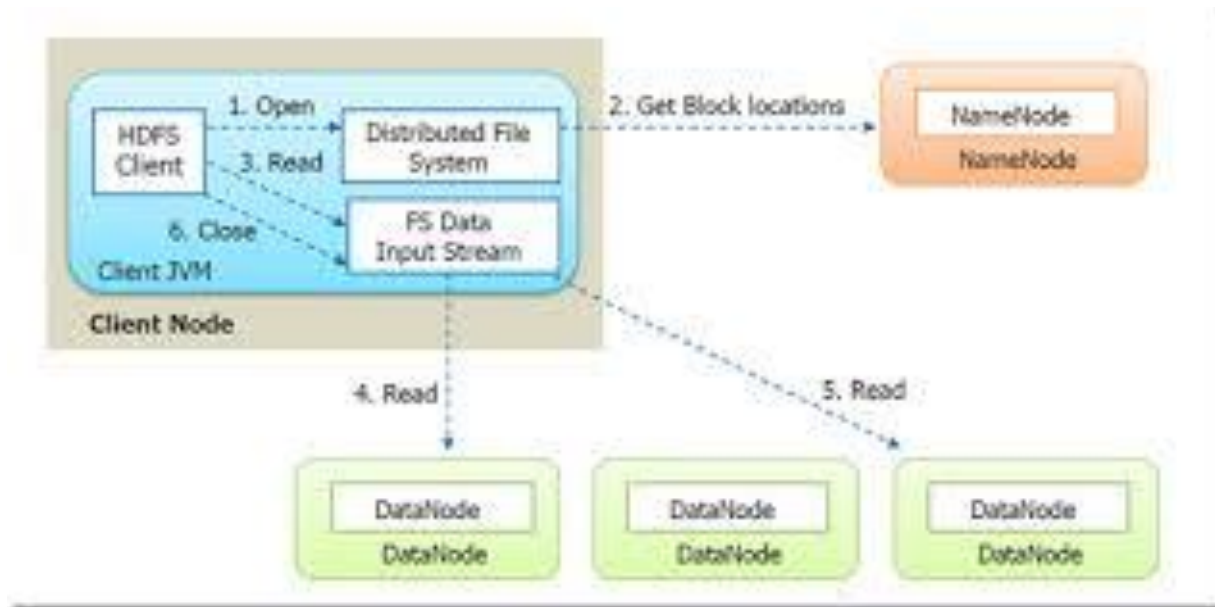
Read Data Mechanism

- Scenario: Open /user/data/file.txt
- Step 1: Client contacts NameNode using open() function.
 - It sends an RPC to NameNode asking: “Where are the blocks for file.txt located?”
 - NameNode responds with block IDs + list of DataNodes that have each block.



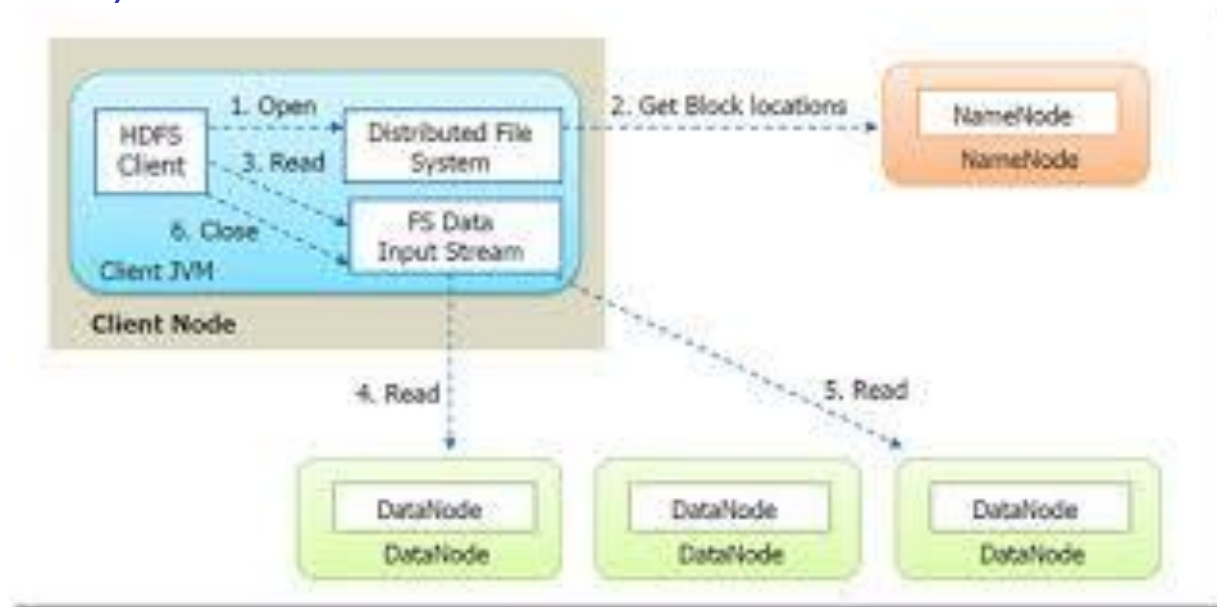
Read Data Mechanism

- Scenario: Open /user/data/file.txt
- Step 2: **Client contacts** nearest DataNode.
 - It picks the DataNode with the shortest network path (rack awareness).
 - It sends an HTTP/DFS read request to get the first block.
 -



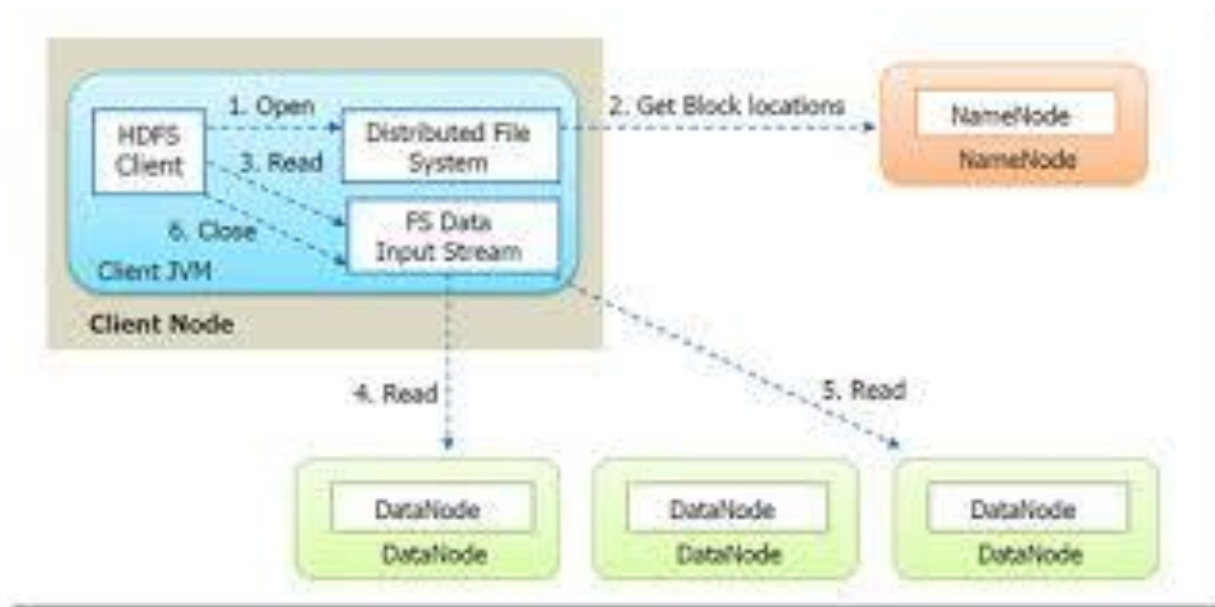
Read Data Mechanism

- Scenario: Open /user/data/file.txt
- Step 3: DataNode streams the block's bytes directly to the client.
 - Note: the nearest DataNode is considered at first.
 - The read() operation is repeatedly streamed to the client.
 - Also, Client verifies checksum.



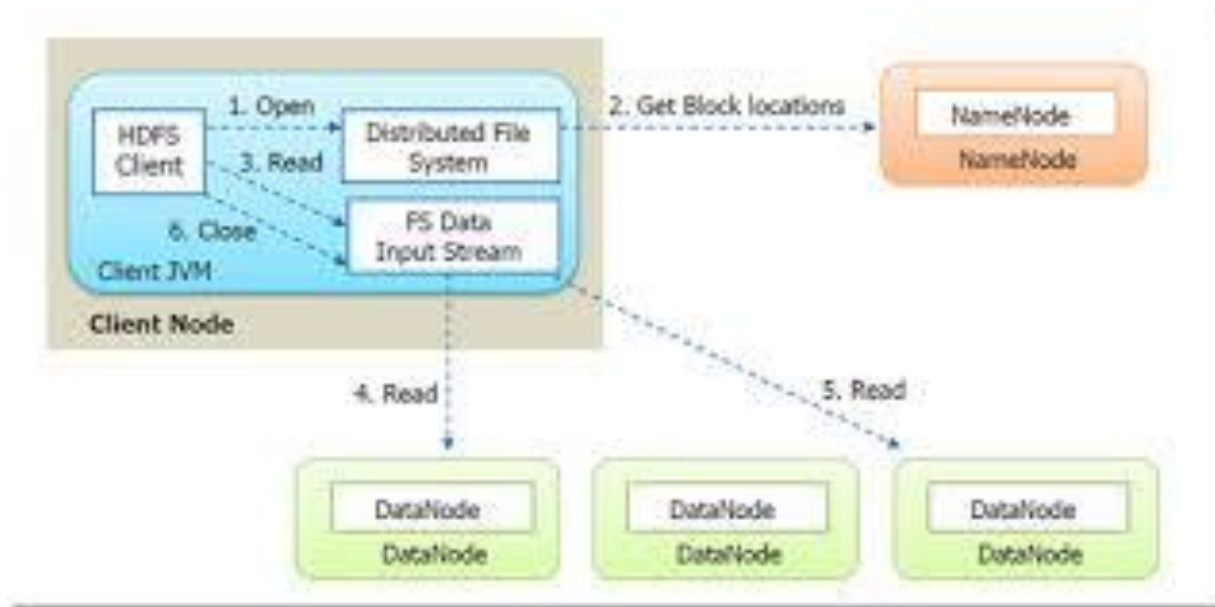
Read Data Mechanism

- Scenario: Open /user/data/file.txt
- Step 4: Once Block 1 is read, client requests Block 2 from the next best DataNode.
 - This continues until the full file is reconstructed on the client side.



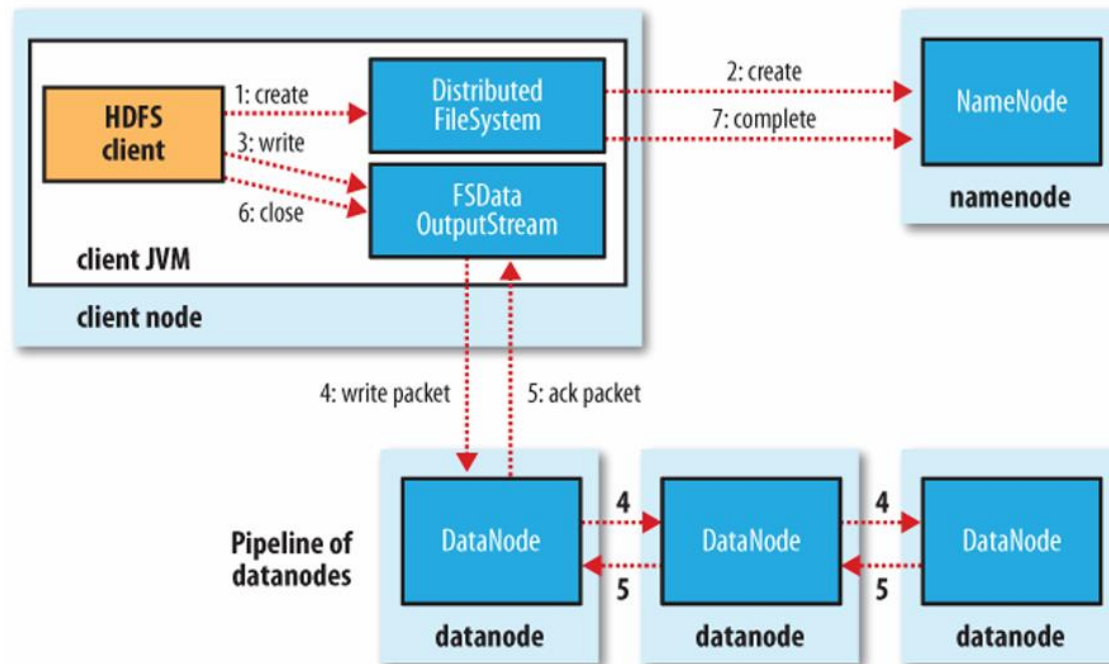
Read Data Mechanism

- Scenario: Open /user/data/file.txt
- Step 5: When the end of the block is reached, DFSInputStream will close the connection to the DataNode.
- Step 6: The client calls close() on FSDataInputStream.
-



Write Data Mechanism

- Scenario: Write /user/data/file.txt
- The files in HDFS are write-once category.
- These files can be appended. But, we cannot modify it in place.
- Step 1: Client requests to create file...

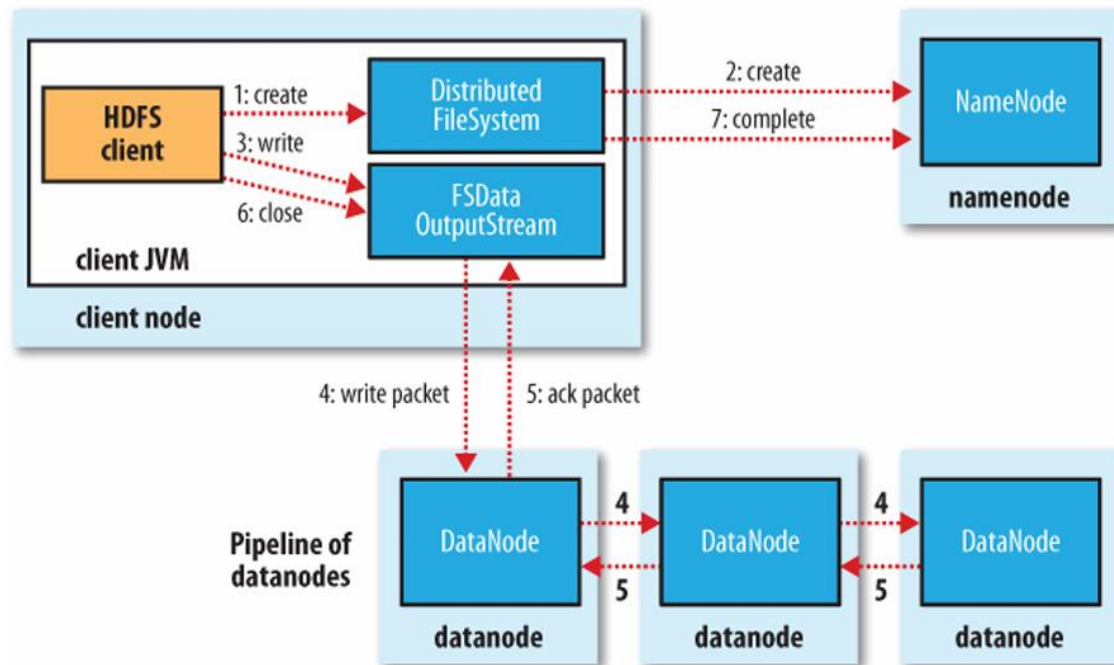


Write Data Mechanism

- Step 2: DistributedFileSystem makes an RPC call to the NameNode to create the file using RPC.
 - NameNode checks if the filename already exists...
 - It checks the write permissions of the client.
 - If allowed, NameNode allocates block IDs and chooses target DataNodes for each block based on replication factor and rack awareness.
 - Also, it responds with the DataNode pipeline if replicas are involved.
 - E.g., for 3 replicas,
 - DataNode 1 -> DataNode 2 -> DataNode 3

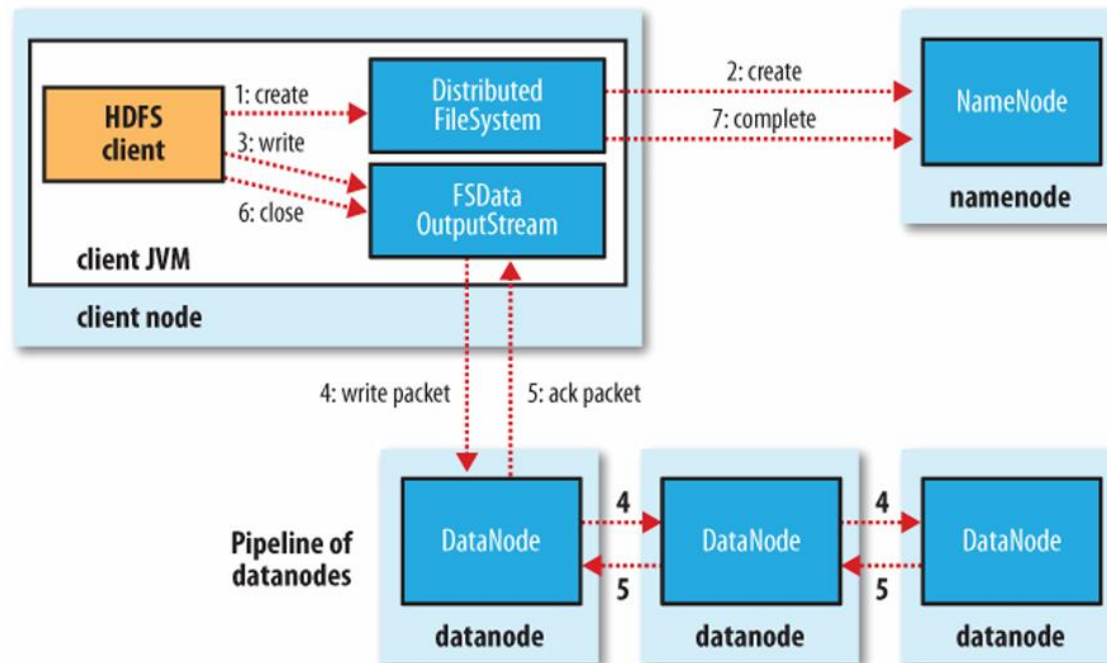
Write Data Mechanism

- Scenario: Write /user/data/file.txt
- Step 3: Client writes to FSDataOutputStream.
 - The FSDataOutputStream splits data into packets.
 - Data is split into packets (smaller chunks, e.g., 64 KB). The packets are written to an internal queue named *data queue*.



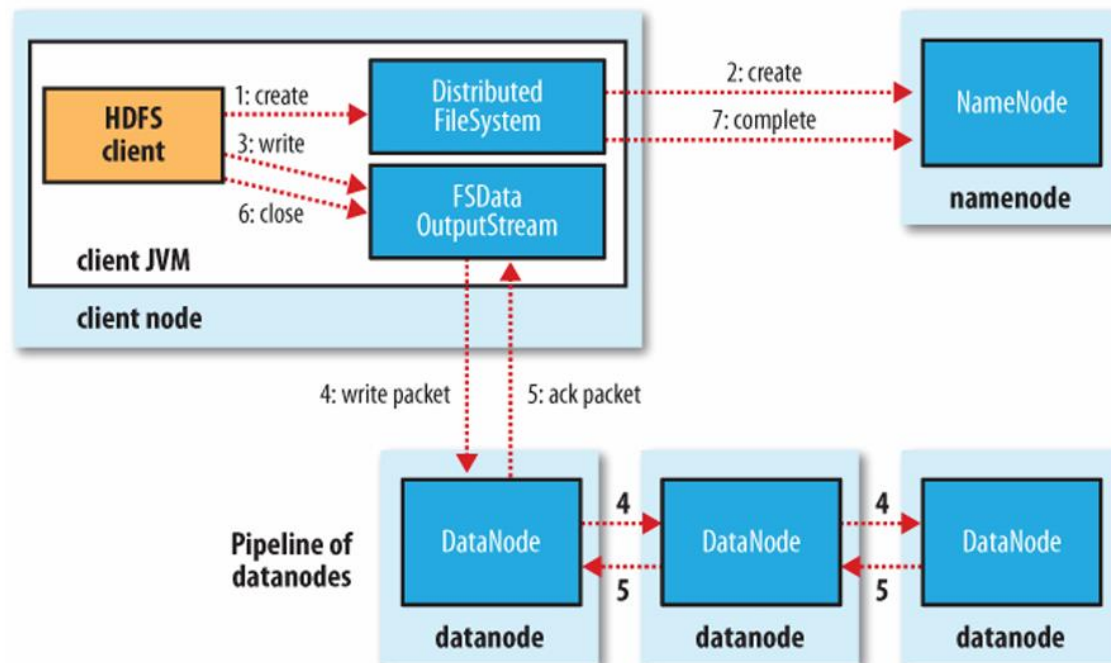
Write Data Mechanism

- Scenario: Write /user/data/file.txt
- Step 4: Packets are sent to the first DataNode in the pipeline using DataStreamer.
 - DataNode1 writes the packet to local disk, then streams it to DataNode2; and, it continues...



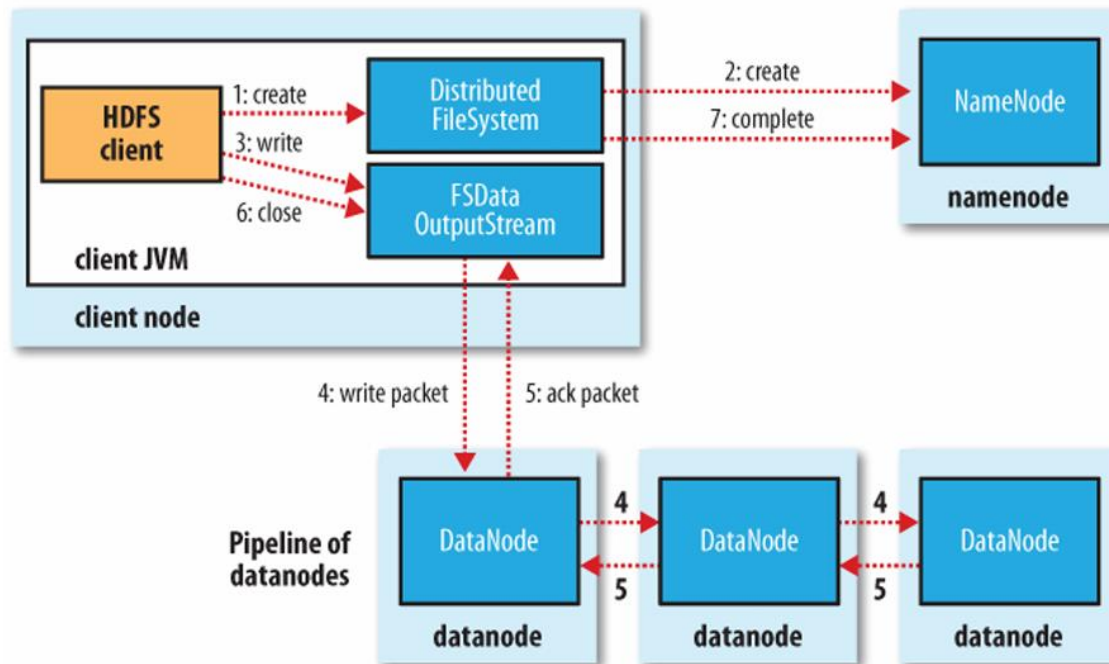
Write Data Mechanism

- Scenario: Write /user/data/file.txt
- Step 5: Acknowledgement Packets are received in return from all DataNodes.
 - The last acknowledgement packet is sent to the client upon the successful write process between the DataNodes.



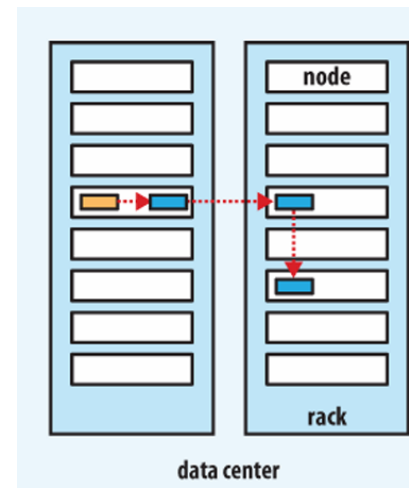
Write Data Mechanism

- Scenario: Write /user/data/file.txt
- Step 6: Finally, the client sends a close() signal to NameNode via. FSDataOutputStream.



Replica Placements

- Tradeoff between
 - RELIABILITY vs. BANDWIDTH
 - For e.g., placing all replicas on a single node
 - Poor reliability; Good Bandwidth
 - For e.g., placing all replicas on various continents
 - Good reliability; Poor Bandwidth
- Typical scenario
 - First replica on the same node;
 - Second replica on the second rack;
 - Third replica on any different server of the second rack.



Popular HDFS File Operations

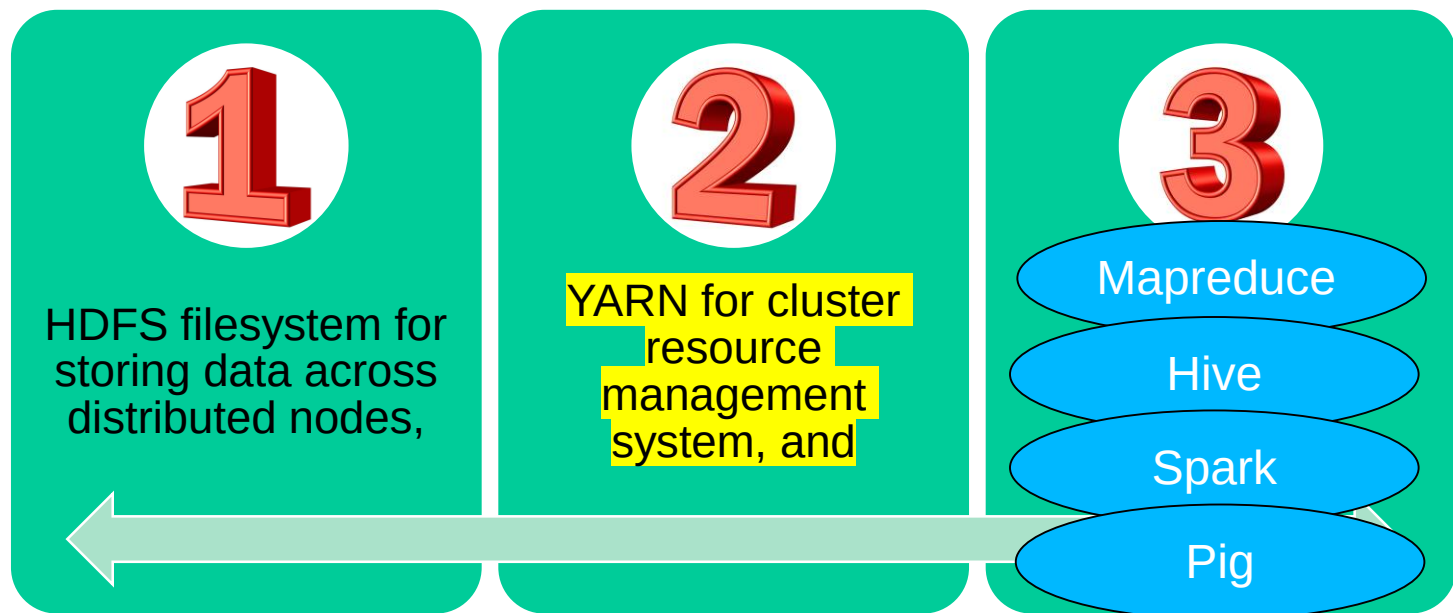
- List all files in HDFS
 - `hdfs dfs -ls /path`
- Create a directory
 - `hdfs dfs -mkdir -p /path/to/dir`
- Upload file from local to HDFS
 - `hdfs dfs -put localfile.txt /hdfs/path/`
- Download file from HDFS to local
 - `hdfs dfs -get /hdfs/path/file.txt localdir/`
- View file contents
 - `hdfs dfs -cat /hdfs/path/file.txt`
- Delete a file
 - `hdfs dfs -rm -r /hdfs/path/to/delete`

Popular HDFS File Operations

- Get the cluster report
 - `hdfs dfsadmin –report`
- Get the safemode status
 - `hdfs dfsadmin -safemode get`
- List running YARN applications
 - `yarn application –list`
- Kill a running YARN application
 - `yarn application -kill <ApplicationID>`

Hadoop Fundamentals

- Hadoop is a software library-based framework that allows distributed processing of large datasets across a large computing cluster.
- It utilizes 3+ components (from Hadoop 2x)...



YARN – A Hadoop Component

- Traditionally, MapReduce attempted to do all job processing and resource orchestration.
- But, Hadoop ecosystem grew further to support multiple types of programming models (including streaming-based applications).
- Hence, YARN (a dedicated resource management system became crucial in the Hadoop ecosystem.
- What is YARN?
 - Yet Another Resource Negotiator
 - It is the Hadoop's cluster resource management system.

Limitations of MapReduce 1 (Birth of YARN)

- **Challenge 1:** Tight Coupling of Resource Management & Processing
- i.e., the JobTracker handled both:
 - Cluster-wide resource management (CPU/memory scheduling).
 - Job scheduling & monitoring.
- Issue:
 - Single point of bottleneck and failure.

Limitations of MapReduce 1 (Birth of YARN)

- **Challenge 2:** Only one programming model (MapReduce)
- Issue:
 - Streaming applications are not possible.
 - Iterative applications are not possible.
 - Hence, a limited scope for improvements.

Limitations of MapReduce 1 (Birth of YARN)

- **Challenge 3: Scalability Bottleneck**
 - One JobTracker had to manage all jobs + resources in the cluster.
 - It could not scale beyond 4000 nodes and 40000 tasks.
- **Challenge 4: Inefficient Resource Utilization**
 - Slots were defined as map slots and reduce slots (fixed per node).
 - Issue: Hence, it leads to underutilized cluster resources.

Limitations of MapReduce 1 (Birth of YARN)

- **Challenge 5:** Poor Multi-Tenancy Support
 - No strong mechanisms for **fair sharing** among multiple users.
 - Multi-tenancy support is not available.

YARN vs. HDFS

- HDFS – The goal is to store huge files across machines using namenode and datanodes.
- YARN – The goal is to allocate cluster resources and schedule jobs.
- YARN (Yet Another Resource Negotiator) is the resource management layer of Hadoop.
- It decouples resource management (YARN's job) from data storage (HDFS's job).

YARN Architecture

- YARN has 4 major components:
 - ResourceManager (RM)
 - NodeManager (NM)
 - ApplicationManager (AM)
 - Containers
- **ResourceManager** = Principal of a college (decides who gets classrooms).
- NodeManager = Department heads (manage classrooms in their department).
- **ApplicationManager** = Course instructor (manages one course, allocates tasks).
- Containers = Classrooms (where actual teaching happens).

YARN Architecture - Components



ResourceManager (RM)

- It is the master daemon running on one node.
- It is responsible for global resource allocation across the cluster.



YARN Architecture - Components

ResourceManager
(RM)
(Sub-components)

- **Scheduler:**
 - Allocates containers (CPU + memory units) based on policies (FIFO, Capacity, Fair).
 - It does not monitor tasks or track progress.
- **ApplicationManager**
 - It accepts job submissions.
 - It manages the lifecycle of applications (for e.g., start, monitor, finish).
 - It allocates the first container for each job (to run the ApplicationMaster).

NodeManager (NM)

- It runs on every worker node.
- It monitors and reports available resources (CPU, memory, disk, network) to the RM.
- It launches and manages containers on its node.
- It collects and forwards logs, health, and resource usage to RM.

ApplicationMaster (AM)

- It is a per-application master process.
- It is launched in the first container.
- It is specific to each application type (e.g., MapReduce AM, Spark AM).
- The major responsibilities include:
 - It negotiate resources with RM.
 - It works with NodeManagers to run tasks in allocated containers.
 - It handles application logic (task retry, failure handling).
 - It dies once the application finishes.

Containers

- Containers are the execution units in YARN (bundle of CPU cores + RAM allocated).
- Every task (MapReduce task, Spark executor, etc.) runs inside a container.
- Containers are managed by NodeManagers.

YARN – Interaction Steps – Job LifeCycle

Step 1: Job Submission

The user (client) submits an application to the ResourceManager (RM).

Step 2: Launch ApplicationManager

RM allocates the first container for the job.
The job-specific ApplicationMaster starts inside this container.

Step 3 - Resource Negotiation

The AM communicates with RM to request resources (more containers).

YARN – Interaction Steps – Job LifeCycle

Step 4: Task Execution

RM grants containers.

The AM instructs
NodeManagers to launch
containers.

Actual tasks run inside these
containers.

Step 5: Monitoring & Completion

AM monitors progress, retries
failed tasks if needed.

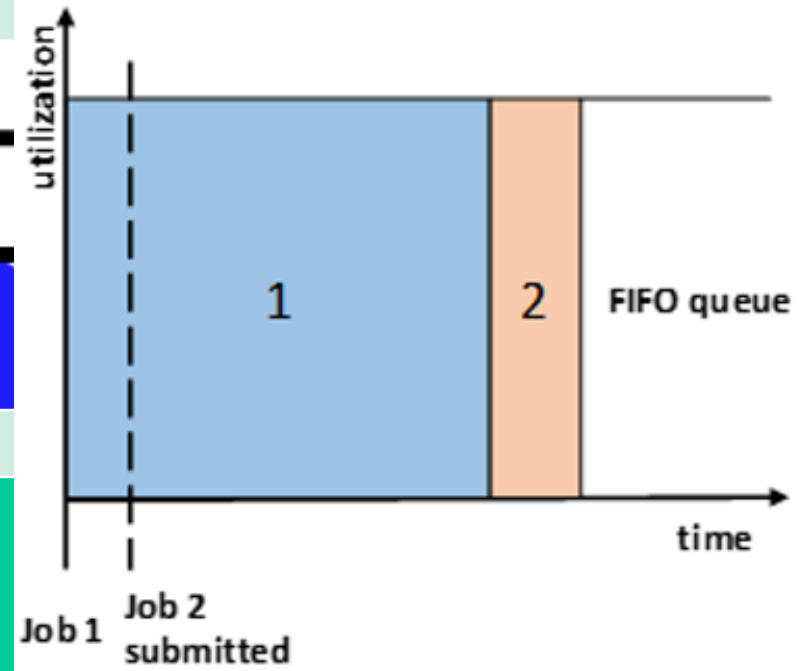
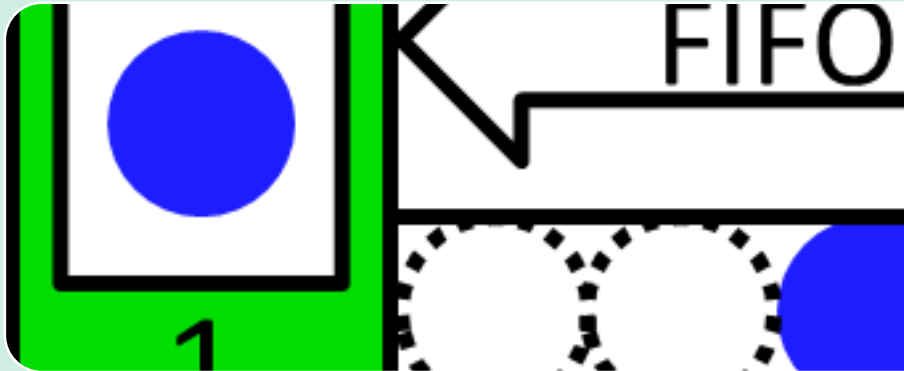
Once job is done, AM shuts
down.

RM releases resources for
other jobs.

YARN Scheduling

- In an ideal world, all requests that a YARN application makes would be granted immediately.
- In a real-world, resources are limited. Hence, schedulers and configurable policies are utilized.
- 3 schedulers (policies) are available:
 - FIFO scheduler
 - Capacity scheduler
 - Fair scheduler

YARN Scheduling Policies - FIFO

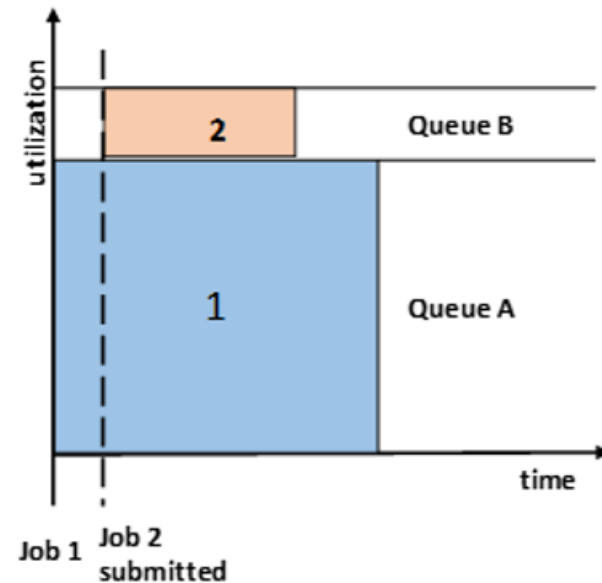


FIFO Scheduler

- First In, First Out queue.
- Jobs are executed in the order they arrive.
- Simple, but unfair in multi-user environments.

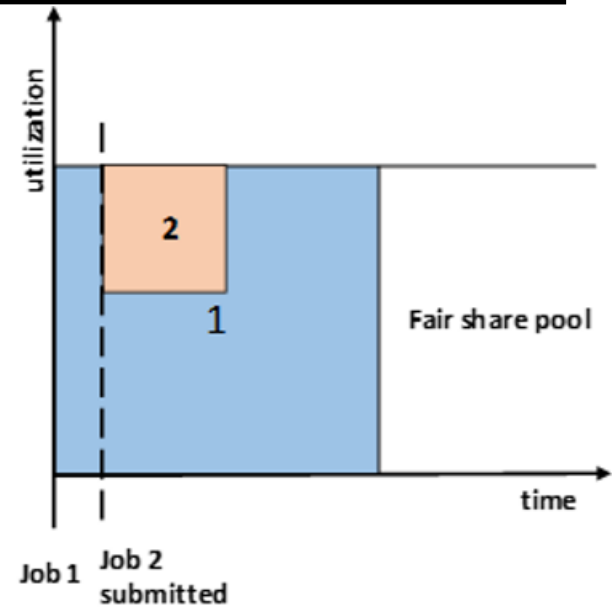
YARN Scheduling Policies - Capacity

- Capacity Scheduler
 - Here, the resources are divided into multiple queues, each with a guaranteed capacity (% of cluster).
- Example:
 - Queue A (Research) → 60% of cluster
 - Queue B (Teaching) → 40% of cluster
 - If one queue is idle, other queues can temporarily borrow resources.
- This scheduler is good for multi-tenant clusters (e.g., university, enterprise).



YARN Scheduling Policies – Fair

- Fair Scheduler
 - It ensures all running jobs get a fair share of resources over time.
- Example:
 - If 2 jobs are running (assume that each job gets 50% resources).
 - If 3 jobs (assume that each job gets ~33% resources)
 - And, so forth...
 - It prioritizes small jobs (doesn't let big jobs starve them).
 - It is used in production Spark clusters often.



YARN command

- yarn cluster –list
 - Display running applications and cluster state.
- yarn node –list –all
 - List all NodeManagers in the cluster with the states (active, unhealthy, decommissioned).
- yarn node –status <NodeID>
- yarn application –list
 - Displays currently running applns.
- yarn application –kill <AppID>
 - Terminate an application

YARN command

- yarn application –status <AppID>
- yarn container –status <ContainerID>
- yarn container –list <NodeID>
 - Shows running containers for a specific NodeManager.
- yarn logs –applicationID <AppID>
 - Fetch logs of an application from all containers
- Yarn logs –applicationID <AppID> - containerID <ContainerID>
 - Fetch logs of an application from a specific container.

Hadoop Demo and Installation Steps: (Ubuntu)

1. sudo apt update
2. Install synaptic
3. Install jdk java, ssh, and pdsh
4. Set JAVA_HOME and restart the machine.
5. Download <https://dlcdn.apache.org/hadoop/common/stable/>
6. Set JAVA environment in file etc/hadoop/hadoop-env.sh
7. #For Hadoop

export

HADOOP_HOME=/home/xxxxxx/xxxxxxxxx/workspace/Eclipseworkspace/hadoop/hadoop-3.4.1

export HADOOP_MAPRED_HOME=\$HADOOP_HOME

export HADOOP_COMMON_HOME=\$HADOOP_HOME

export HADOOP_HDFS_HOME=\$HADOOP_HOME

export YARN_HOME=\$HADOOP_HOME

export HADOOP_COMMON_LIB_NATIVE_DIR=\$HADOOP_HOME/lib/native

export PATH=\$PATH:\$HADOOP_HOME/sbin:\$HADOOP_HOME/bin

export LD_LIBRARY_PATH=\$LD_LIBRARY_PATH:\$HADOOP_HOME/lib/native

export HADOOP_INSTALL=\$HADOOP_HOME

export HADOOP_CONF_DIR=\$HADOOP_HOME/etc/hadoop

Hadoop Demo and Installation Steps: (Ubuntu)

```
iotcloud@iotcloud-Precision-Tower-7810:~/shaju/workspace/Eclipseworkspace/hadoop/hadoop-3.4.1/etc/hadoop$ hadoop
```

```
Usage: hadoop [OPTIONS] SUBCOMMAND [SUBCOMMAND OPTIONS]
```

```
or hadoop [OPTIONS] CLASSNAME [CLASSNAME OPTIONS]
```

```
where CLASSNAME is a user-provided Java class
```

```
OPTIONS is none or any of:
```

buildpaths	attempt to add class files from build tree
--config dir	Hadoop config directory
--debug	turn on shell script debug mode
--help	usage information
hostnames list[,of,host,names]	hosts to use in worker mode
hosts filename	list of hosts to use in worker mode
loglevel level	set the log4j level for this command
workers	turn on worker mode

```
SUBCOMMAND is one of:
```

```
Admin Commands:
```

```
daemonlog    get/set the log level for each daemon
```

```
Client Commands:
```

archive	create a Hadoop archive
checknative	check native Hadoop and compression libraries availability
classpath	prints the class path needed to get the Hadoop jar and the required libraries
conftest	validate configuration XML files
credential	interact with credential providers
distch	distributed metadata changer

Hadoop Demo and Installation Steps: (Ubuntu)

```
ssh localhost  
ssh-keygen -t rsa -P "" -f ~/.ssh/id_rsa  
cat ~/.ssh/id_rsa.pub >> ~/.ssh/authorized_keys  
chmod 0600 ~/.ssh/authorized_keys
```

Hadoop Demo and Installation Steps: (Ubuntu)

```
core-site.xml x hdfs-site.xml
c > hadoop > core-site.xml
1  <?xml version="1.0" encoding="UTF-8"?>
2  <?xml-stylesheet type="text/xsl" href="configuration.xsl"?>
3  <!--
4      Licensed under the Apache License, Version 2.0 (the "License");
5      you may not use this file except in compliance with the License.
6      You may obtain a copy of the License at
7
8      http://www.apache.org/licenses/LICENSE-2.0
9
10     Unless required by applicable law or agreed to in writing, software
11     distributed under the License is distributed on an "AS IS" BASIS,
12     WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
13     See the License for the specific language governing permissions and
14     limitations under the License. See accompanying LICENSE file.
15 -->
16
17 <!-- Put site-specific property overrides in this file. -->
18
19 <configuration>
20   <property>
21     <name>fs.defaultFS</name>
22     <value>hdfs://localhost:9000</value>
23   </property>
24 </configuration>
25
```

Hadoop Demo and Installation Steps: (Ubuntu)

```
core-site.xml  hdfs-site.xml ×
c > hadoop > hdfs-site.xml
1  <?xml version="1.0" encoding="UTF-8"?>
2  <?xml-stylesheet type="text/xsl" href="configuration.xsl"?>
3  <!--
4      Licensed under the Apache License, Version 2.0 (the "License");
5      you may not use this file except in compliance with the License.
6      You may obtain a copy of the License at
7
8      http://www.apache.org/licenses/LICENSE-2.0
9
10     Unless required by applicable law or agreed to in writing, software
11     distributed under the License is distributed on an "AS IS" BASIS,
12     WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
13     See the License for the specific language governing permissions and
14     limitations under the License. See accompanying LICENSE file.
15 -->
16
17 <!-- Put site-specific property overrides in this file. -->
18
19 <configuration>
20   <property>
21     <name>dfs.replication</name>
22     <value>1</value>
23   </property>
24 </configuration>
```


Hadoop Demo and Installation Steps: (Ubuntu)

Format hdfs:

```
bin/hdfs namenode -format
```

Start NameNode and DataNode daemons:

```
sbin/start-dfs.sh
```

Hadoop Demo and Installation Steps: (Ubuntu)

http://localhost:9870/dfshealth.html#tab-overview

Summary

Security is off.

Safemode is off.

1 files and directories, 0 blocks (0 replicated blocks, 0 erasure coded block groups) = 1 total filesystem object(s).

Heap Memory used 276.98 MB of 456 MB Heap Memory. Max Heap Memory is 15.66 GB.

Non Heap Memory used 54.97 MB of 58.06 MB Committed Non Heap Memory. Max Non Heap Memory is <unbou

Configured Capacity:	438.55 GB
Configured Remote Capacity:	0 B
DFS Used:	24 KB (0%)
Non DFS Used:	399.69 GB
DFS Remaining:	16.51 GB (
Block Pool Used:	24 KB (0%)
DataNodes usages% (Min/Median/Max/stdDev):	0.00% / 0.0
Live Nodes	1 (Decomr
Dead Nodes	0 (Decomr
Decommissioning Nodes	0
Entering Maintenance Nodes	0
Total Datanode Volume Failures	0 (0 B)
Number of Under-Replicated Blocks	0

Hadoop Demo and Installation Steps: (Ubuntu)

Next: Create a HDFS directories to execute MapReduce jobs:

```
bin/hdfs dfs -mkdir -p /user/<username>
```

Next: Copy input files:

```
bin/hdfs dfs -mkdir input
```

```
bin/hdfs dfs -put etc/hadoop/*.xml input
```

Next: Run examples:

```
bin/hadoop jar share/hadoop/mapreduce/hadoop-mapreduce-examples-3.4.1.jar grep input output 'dfs[a-z.]+'
```

Next: Examine output files:

```
bin/hdfs dfs -get output output
```

```
cat output/*
```

Next: Stop the daemons:

```
sbin/stop-dfs.sh
```